

Certificate in Quantitative Finance

Final Project Brief

You are about to start the Certificate in Quantitative Finance (CQF) exam. By accessing the exam materials, you acknowledge and agree to the CQF Terms of Use (<https://www.fitch.group/terms-of-use/>) and the following terms:

Fitch Learning Limited ("Fitch Learning") holds ownership or licenses for all intellectual property rights related to the www.cqf.com website and any content available on the website and the exam, including text, graphics, software, photographs, images, videos, sound, trademarks, and logos (the "CQF Materials").

By clicking "Start Assignment," you consent not to infringe upon Fitch Learning's intellectual property rights. You agree not to copy, modify, reproduce, publish, sell, upload, broadcast, post, transmit, license, distribute, adapt, merge, translate, disassemble, decompile, or reverse engineer any part of the CQF Materials without Fitch Learning's written consent.

Fitch Learning reserves the right to take action against any unauthorized use of the CQF Materials in the event of a breach of these terms. This may include but is not limited to: (i) disabling and terminating your account; (ii) requesting you to remove materials that breach these terms; and/or (iii) commencing legal proceedings to protect Fitch Learning's or its licensors' intellectual property rights.

June 2025 Cohort

This document outlines topics available for this cohort. No other topics can be submitted. Each topic has by-step instructions to give you a structure (not limit) as to what and how to implement.

Marks earned will strongly depend on your coding of numerical techniques and presentation of how you explored and tested a quantitative model (report in PDF or HTML). Certain numerical methods are too involved or auxiliary to the model, for example, do not recode optimisation or RNs generation. Code adoption allowed if the code fully modified by yourself.

A capstone project requires own study and ability to work with documentation on packages that implement numerical methods in your coding environment e.g., Python, R, Matlab, C#, C++, Java. You do not need to pre-approve the coding language and use of libraries, including very specialised tools such as Scala, kdb+ and q. However, software like EViews is not coding.

Exclusively for current CQF delegates. No distribution.

To complete the project, you must code the model(s) and its numerical techniques from one topic from the below options and write an analytical report. If you continue from a previous cohort, please review topic description because tasks are regularly reviewed. It is not possible to submit past topics.

1. Credit Spread for a Basket Product (CR)
2. Deep Learning for Asset Prediction (DL)
3. Pairs Trading Strategy Design & Back test (TS)
4. Portfolio Construction using Black-Litterman Model and Factors (PC)
5. Delta in a Local Volatility World. Optimal Hedging. (LV)
6. Blending Ensemble for Classification (ML)
7. Algorithmic Trading for Reversion and Trend-Following (AL)
8. Deep Neural Networks for Solving High Dimensional PDEs (DN)

Topics List for the current cohort will be available on the relevant page of Canvass Portal.

Project Report and Submission

- First recommendation: do not submit Python Notebook 'as is' – there is work to be done to transform it into an analytical report. Remove printouts of large tables/output. Write up mathematical sections (with LaTeX markup). Write up analysis and comparison for results and stress-testing (or alike). Explain your plots. Think like a quant about the computational and statistical properties: convergence/accuracy/variance and bias. Make a table of the numerical techniques you coded/utilised.
- Project Report must contain sufficient mathematical model(s), numerical methods and an adequate conclusion discussing pros and cons, further development.
- There is no set number of pages. Some delegates prefer to present multiple plots on one page for comparability, others choose more narrative style.
- It is optimal to save Python Notebook reports as HTML but do include a PDF with page numbers – for markers to refer to.
- Code must be submitted and working.

FILE 1. For our download and processing scripts to work, it is necessary to name and upload the project report as ONE file (pdf or html) with the two-letter project code, followed by your name as registered on CQF Portal.

Examples: TS John Smith REPORT.pdf or PC Xiao Wang REPORT.pdf

FILE 2. All other files, code and a pdf declaration (if not the front page) must be uploaded as additional ONE zip file, for example TS John Smith CODE.zip. In that zip include converted PDF, Python, and other code files. Do not submit unzipped .py, .cpp files as cloud anti-virus likely to flash red on our side. Do not submit files with generic names, such as CODE.zip, FinalProject.zip, Final Project Declaration.pdf, etc. Such files will be disregarded.

Submission date for the project is Tuesday 20th January 2026, 23.59 GMT

There is no extension time to Final Project.

Projects without a hand-signed declaration or working code are incomplete.

Failure to submit ONE report **file** and ONE zip **file** according to the naming instructions means such a project will miss an allocation for grading.

All projects are checked for originality. We reserve an option of a viva voce before the qualification to be awarded.

Project Support

Advanced Electives

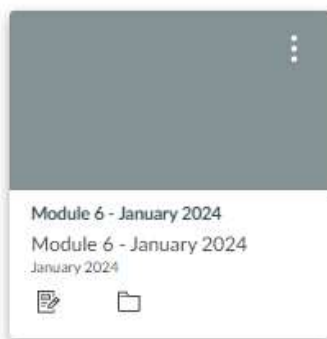
To gain background knowledge in a focused way, we ask you to review two Advanced Electives. Electives canvass knowledge areas and can be reviewed before/at the same time/closer to writing up Analysis and Discussion (explanation of your results).

- There is no immediate match between Project Topics and Electives
- Several workable combinations for each Project Topic are possible.
- One elective learning strategy is to select one 'topical elective' and one 'coding elective.'

To access the electives:

Login to the CQF Learning Hub

Navigate to Module 6 on your Dashboard.



Click on *Electives* button on global navigation menu.



Scroll down to *Electives*, then click the *Electives Catalog*.



You will be redirected to the electives Catalogue, where you can view and review all electives available to you. Full descriptions for each elective can be found [here](#).



When on an elective click the *enrol* button



You will see the confirmation page, click the *enrol in Course* button to confirm your selection.

You will land on the successful enrolment page, where you can click to start the elective or return to the catalogue page.



When on the catalogue page you can click the *Learning Platform* link to return to Canvas. Your electives selected will appear on your learning dashboard.

Workshop & Tutorials

Each project title is supported by a faculty member alongside a set of project workshops and tutorials.

DATE	TITLE	TIME
29/11/2025	Final Project Workshop I (CR, PC & LV)	13:00 – 16:30 GMT
06/12/2025	Final Project Workshop II (TS, DL, ML, AL, DN & LV	13:00 – 16:30 GMT
08/12/2025	Final Project Tutorial I (CR topic)	18:00 – 19:30 GMT
10/12/2025	Final Project Tutorial II (DN & PC topic)	18:00 – 19:30 GMT
15/12/2025	Final Project Tutorial III (TS, LV & AL topic)	18:00 – 19:30 GMT
18/12/2025	Final Project Tutorial IV (DL & ML topic)	18:00 – 19:30 GMT

Faculty Support

Lead: Riaz Ahmad

Title: Credit Spread for a Basket Product (CR)

Project Code: CR

Lead: Kannan Singaravelu

Title: Deep Learning for Asset Prediction (DL)

Ensemble for Classification (ML)

Project Code: DL & ML

Lead: Richard Diamond

Title: Pairs Trading Strategy Design & Backtest (TS)

Delta in a Local Volatility World. Optimal Hedging (LV)

Algorithmic Trading for Reversion and Trend-Following (AL)

Project Code: TS, LV, AL

Lead: Panos Parpas

Title: Portfolio Construction using Black-Litterman Model and Factors

Deep Neural Networks for Solving High Dimensional PDEs (DN)

Project Code: PC & DN

To ask faculty a question on your chosen topic, please submit a support ticket by clicking on the Support button which can be found in the bottom hand right corner on your portal.

Coding for Quant Finance

- Choose programming environment that has appropriate strengths and facilities to implement the topic (pricing model). Common choice is Python, Java, C++, R, Matlab. Exercise judgement as a quant: which language has libraries to allow you to code faster, validate easier.
- Use of R/Matlab/Mathematica is encouraged. Often there a specific library in Matlab/R gives fast solution for specific models in robust covariance matrix/cointegration analysis tasks.
- Project Brief give links to nice demonstrations in Matlab, and Webex sessions demonstrate Python notebooks {does not mean your project to be based on that ready code.
- Python with pandas, matplotlib, sklearn, and tensorflow forms a considerable challenge to Matlab, even for visualization. Matlab plots editor is clunky, and it is not that difficult to learn various plots in Python.
- 'Scripted solution' means the ready functionality from toolboxes and libraries is called, but the amount of own coding of numerical methods is minimal or non-existent. This particularly applies to Matlab/R.
- Projects done using Excel spreadsheet functions only are not robust, notoriously slow and do not give understanding of the underlying numerical methods. CQF-supplied Excel spreadsheets are a starting point and help to validate results but coding of numerical techniques/use of industry code libraries is expected.
- The aim of the project is to enable you to code numerical methods and develop model prototypes in a production environment. Spreadsheets-only or scripted solutions are below the expected standard for completion of the project.
- What should I code? Delegates are expected to re-code numerical methods that are central to the model and exercise judgement in identifying them. Balanced use of libraries is at own discretion as a quant.
- Produce a small table in report that lists methods you implemented/adjusted. If using ready functions/borrowed code for a technique, indicate this and describe the limitations of numerical method implemented in that code/standard library.

- It is up to delegates to develop their own test cases, sensibility checks and validation. It is normal to observe irregularities when the model is implemented on real life data. If in doubt, reflect on the issue in the project report.
- The code must be thoroughly tested and well-documented: each function must be described, and comments must be used. Provide instructions on how to run the code.

This page intentionally left blank.

Credit Spread for a Basket Product

Price a fair spread for a portfolio of CDS for 5 reference names (Basket CDS), as an expectation over the joint distribution of default times. The distribution is unknown analytically and so, co-dependent uniform variables are sampled from a copula and then converted to default times using a marginal term structure of hazard rates (separately for each name). Copula is calibrated by estimating the appropriate default correlation (historical data of CDS differences is natural candidate but poses market noise issue). Initial results are histograms (uniformity checks) and scatter plots (co-dependence checks). Substantial result is sensitivity analysis by repricing.

A successful project will implement sampling from both, Gaussian and t copulae, and price all k-th to default instruments (1st to 5th). Spread convergence can require the low discrepancy sequences (e.g., Halton, Sobol) when sampling. Sensitivity analysis *wrt* inputs is required.

Data Requirements

Two **separate** datasets required, together with matching discounting curve data for each.

1. **A snapshot of credit curves** on a particular day. A debt issuer likely to have a USD/EUR CDS curve – from which a term structure of hazard rates is bootstrapped and utilised to obtain exact default times, $u_i \rightarrow \tau_i$. In absence of data, spread values for each tenor can be assumed or stripped visually from the plots in financial media. The typical credit curve is concave (positive slope), monotonically increasing for 1Y, 2Y, ..., 5Y tenors.
2. **Historical credit spreads time series** taken at the most liquid tenor 5Y for each reference name. Therefore, for five names, one computes 5×5 default correlation matrix. Choosing corporate names, it is much easier to compute correlation matrix from equity returns.

Corporate credit spreads are unlikely to be in open access; they can be obtained from Bloomberg or Reuters terminals (via your firm or a colleague). For sovereign credit spreads, time series of ready bootstrapped PD_{5Y} were available from DB Research, however, the open access varies. Explore data sources such as www.datagrapple.com and www.quandl.com.

Even if CDS_{5Y} and PD_{5Y} series are available with daily frequency, the co-movement of daily changes is market noise *more* than correlation of default events, which are rare to observe. Weekly/monthly changes give more appropriate input for default correlation, however that entails using 2-3 years of historical data given that we need at least 100 data points to estimate correlation with the degree of significance.

If access to historical credit spreads poses a problem remember, default correlation matrix can be estimated from historic equity returns or debt yields.

Step-by-Step Instructions

1. For each reference name, bootstrap implied default probabilities from quoted CDS and convert them to a term structure of hazard rates, $\tau \sim \text{Exp}(\hat{\lambda}_{1Y}, \dots, \hat{\lambda}_{5Y})$.
2. Estimate default correlation matrices (near and rank) and d.f. parameter (ie, calibrate copulae). You will need to implement pricing by Gaussian and t copulae separately.
3. Using sampling from copula algorithm, repeat the following routine (simulation):
 - (a) Generate a vector of correlated uniform random variable.
 - (b) For each reference name, use its term structure of hazard rates to calculate exact time of default (or use semi-annual accrual).
 - (c) Calculate the discounted values of premium and default legs for every instrument from 1st to 5th-to-default. Conduct MC separately or use one big simulated dataset.
4. Average premium and default legs across simulations separately. Calculate the fair spread.

Model Validation

- The fair spread for k th-to-default Basket CDS should be less than $k-1$ to default. Why?
- Project Report on this topic should have a section on **Risk and Sensitivity Analysis** of the fair spread *w.r.t.*
 1. default correlation among reference names: either stress-test by constant high/low correlation or $\pm$ percentage change in correlation from the actual estimated levels.
 2. credit quality of each individual name (change in credit spread, credit delta) as well as recovery rate.

Make sure you discuss and compare sensitivities for all five instruments.

- Ensure that you explain historical sampling of default correlation matrix and copula fit (uniformity of pseudo-samples) – that is, Correlations Experiment and Distribution Fitting Experiment as will be described at the Project Workshop. Use histograms.

Copula, CDF and Tails for Market Risk

The recent practical tutorial on using copula to generate correlated samples is available at: <https://www.mathworks.com/help/stats/copulas-generate-correlated-samples.html>

Semi-parametric CDF fitting gives us percentile values with fitting the middle and tails. Generalised Pareto Distribution applied to model the tails, while the CDF interior is Gaussian kernel-smoothed. The approach comes from Extreme Value Theory that suggests correction for an Empirical CDF (kernel fitted) because of the tail exceedances.

<http://uk.mathworks.com/help/econ/examples/using-extreme-value-theory-and-copulas-to-evaluate-market-risk.html>

<http://uk.mathworks.com/help/stats/examples/nonparametric-estimates-of-cumulative-distribution-functions-and-their-inverses.html>

Deep Learning for Asset Prediction

Summary

Trend prediction has drawn a lot of research for many decades using both statistical and computing approaches including machine learning techniques. Trend prediction is valuable for investment management as accurate prediction could ensure asset managers outperform the market. Trend prediction remains a challenging task due to the semi-strong form of market efficiency, high noise-to-signal ratio, and the multitude of factors that affect asset prices including, but not limited to the stochastic nature of underlying instruments. However, sequential financial time series can be modeled effectively using sequence modeling approaches like a recurrent neural network.

Objective

Your objective is to produce a model to predict positive moves (up trend) using the Long Short-Term Memory Networks. Your proposed solution should be comprehensive with the detailed model architecture, evaluated with a backtest applied to a trading strategy.

- Choose one ticker of your interest from the index, equity, ETF, crypto token, or commodity.
- Predict trend only, for a short-term return (example: daily, 6 hours). Limit prediction to binomial classification: the dependent variable is best labelled $[0, 1]$. Avoid using $[-1, 1]$ as class labels.
- Analysis should be comprehensive with detailed feature engineering, data pre-processing, model building, and evaluation.

Note: You are free to make study design choices to make the task achievable. You may redefine the task and predict the momentum sign (vs return sign) or direction of volatility. Limit your exploration to ONLY one asset. At each step, the process followed should be expanded and explained in detail. Merely presenting python codes without a proper explanation shall not be accepted. The report should present the study in a detailed manner with a proper conclusion. Code reproducibility is a must and the use of modular programming approaches is recommended. Under this topic, you do not recode existing indicators, libraries, or optimization to compute neural network weights and biases.

Step-by-Step Instructions

1. The problem statement should be explicitly specified without any ambiguity including the selection of underlying assets, datasets, timeframe, and frequency of data used.
 - If predicting short-term return signs (for the daily move), then training and testing over up to 5 years should be sufficient. If you attempt the prediction of 5D, 10D return for equity or 1W, 1M for the Fama French factor, you'll have to increase the data required to at least 10 years.
2. Perform exhaustive Feature Engineering (FE).
 - FE should be detailed including the listing of derived features and specification of the target/label. Devise your approach on how to categorize extremely small near-zero returns (drop from the training sample or group with positive/negative returns). The threshold will strongly depend on your ticker. Example: small positive returns below 0.25% can be labelled as negative.
 - Class imbalances should be addressed - either through model parameters or via label definition.
 - Use of features from cointegrated pairs and across assets is permitted but should be tactical about design. There is no one recommended set of features for all assets; however, the initial feature set should be sufficiently large. Financial ratios, advanced technical indicators including volatility estimators, and volume information can be a predictor for price direction.
 - OPTIONAL Use of news heatmap, credit spreads (CDS), historical data for financial ratios, history of dividends, purchases/disposals by key stakeholders (director dealings) or by large funds, or Fama-French factor data can enhance your prediction and can be sourced from your professional subscription.
3. Conduct a detailed Exploratory Data Analysis (EDA).
 - EDA helps in dimensionality reduction via a better understanding of relationships between features and uncovers underlying structure, and invites detection/explanation of the outliers. The choice of feature scaling techniques should be determined by EDA.
4. Proper handling of data is a must. The use of a different set of features, lookback length, and datasets warrant cleaning and/or imputation.
5. Feature transformation should be applied based on EDA.
 - Multi-collinearity analysis should be performed among predictors.
 - Multi-scatter plots presenting relationships among features are always a good idea.
 - Large feature sets (including repeated kinds, and different lookbacks) warrant a reduction in dimensionality in features. Self Organizing Maps (SOM), K-Means clustering, or other methods can be used for dimensionality reduction. Avoid using Principal Component Analysis (PCA) for non-linear datasets/predictors.

6. Perform extensive and exhaustive model building.

- Design the neural network architecture after extensive and exhaustive study.
- The best model should be presented only after performing the hyperparameter optimization and compared with the baseline model.
- The choice and number of hyperparameters to be optimized for the best model are design choices. Use experiment trackers like MLFlow or TensorBoard to present your study.

7. The performance of your proposed classifier should be evaluated using multiple metrics including backtesting of the predicted signal applied to a trading strategy.

- Investigate the prediction quality using AUC, confusion matrix, and classification report including balanced accuracy (if required).
- Predicted signals should be evaluated by applying them to a trading strategy.

* * *

Pairs Trading Strategy Design & Backtest v2025

Cointegrated relationship between prices opens way to structure an arbitrage around the mean-reversion in the special residual. We are equipped with stationarity tests (ADF, KPSS, AZ), and we extend estimation Procedures with mean-reversion. It is normal to start with statistical tests over one period and you can find statistically-confirmed cointegration, but aim for signal generation and backtest (multiple periods, improve spread's mean-reversion). Initial search for pairs can be done with correlation, however, trading with 100%, -100% weights is not conducive to the residual's stationarity. Asset prices must be properly tied in though the Error-Correction Model (EG Procedure).

The numerical techniques to implement: matrix form autoregression, EG Procedure (two steps), mean-reversion evaluation (theta, half-life), and optimising Z^* at least iteratively. You are encouraged to venture into Multivariate Cointegration (Johansen Procedure) and robustness of weights by adaptive estimation. Multivariate can offer you new trading possibilities, at a certain loss in explanation.

Signal Generation and Backtesting

- Consider economic, quant and even-driven reasons for the cointegration to persist. For example, a company and its potential acquisition target are particularly likely to form a robust coint relationship. Be inventive beyond equity pairs, other pairs can be currencies, US Treasury / UK Gilt yields, commodity futures, VIX futures.
- There are studies that filter the entire market for cointegration, but you can take more targeted approach, eg use the above, check for high correlation in search of pairs.
- Arb is realised by making entry/exit decisions with β_{Coint} as allocations w . This is a natural long-short portfolio that generates a mean-reverting spread. All projects should include (a) trading signal generation from OU process fitting and (b) backtesting techniques, typically drawdown plots, rolling SR, and rolling beta.
- Does cumulative P&L behave as expected for a statistical arb trade? Is P&L coming from a few or many trades, what is the half-life? Maximum Drawdown and behaviour of volatility/VaR?

Step-by-Step Instructions

Part I: Pairs trade design. Preparatory cointegration analysis

1. Recode regression estimation in matrix form as an exercise. Can implement Vector Autoregression specification tests, which are (a) stability check with eigenvalues, and (b) identifying optimal lag p with AIC BIC tests. However, these apply only for structural models between stationary changes; we are not forecasting returns in this project.
2. Implement Engle-Granger procedure and run it for each pair – later you can split the dataset into several periods and present more dynamic results. For Step 1, use the Augmented DF with lag=1. For Step 2, write a short analysis, eg analyse the significance of EC-term, has it stayed the same each period.
3. We extend the procedure with 'Step 3' (not in the original Engle-Granger) mean-reversion evaluation. That allows trade design: enter on bounds $\mu_e \pm Z\sigma_{eq}$ and exit on e_t reverting to the level μ_e .

4. Do not assume $Z = 1$, implement some kind of optimisation, or simple iterative choice: vary Z upwards and downwards in increments – and produce a chart or table analysing N_{trades} for each level of Z^* . This is done to explicate the trade-off: wider bounds give the highest cumulative P&L with low N_{trades} , however those trades will be when e_t deviates well-above or -below the level μ_e . That itself might signal the likelihood of a structural break.
5. The formal investigation of structural breaks, particularly within the reduced rank multivariate relationships, is beyond the scope of project. Testing for structural breaks is more an art than science. However, present a short discussion of your thought: reasons for and how your chosen asset coint relation might break apart.

If comfortable with R, can optionally utilise the ready multivariate analysis (package *urca*) to identify your cointegrated cases, where your assets have a term structure. Overall, your study should consider 2-3 different pairs.

Part II: Backtesting

Below items are recommendations, you can implement all or only a part of them as suitable to your asset classes and pairs. Overall, a 2-year period is considered to be minimum backtest, subject to the realities of cointegrated situation.

6. Think of ML/scikit-learn inspired backtesting: splitting train/test subsets and other cross-validation as feasible for financial time series (FTS).
7. Perform the systematic backtesting of your trading strategy. Take returns from a pairs trade (already with Z^*): produce drawdown plots, rolling Sharpe Ratio, at least one rolling beta with regard to S&P500 excess returns. Discuss your plots.
8. Cointegration analysis has the advantage of offering stable β'_{Coint} : delivering the stationary spread over 3-6 months without the need to be updated. That might not be a realistic assumption for FTS, for example, the industry sector might experience change in valuation multiples. Discuss benefits and disadvantages of regular re-estimation of cointegrated relationships: 5-8 months rolling window shifting by 10-15 days.

OPTIONAL Would you consider Kalman filter/unscented particle filter for adaptive estimation? Kalman filter can be used to simply re-estimate EG Step 1 of naive coint residual adaptively, hence the study will trace the changes to cointegrating weights. can present the scheme of how coint residual regression equation changes under the adaptive estimation.

TS Project Workshop, Cointegration Lecture and Pairs Trading tutorial are your key resources.

Portfolio Construction using Black-Litterman Model and Factors

Summary

Construct a factor-bearing portfolio, compute at least two kinds of optimisation. Within each optimisation, utilise the Black-Litterman model to update allocations with absolute and relative views. Compute optimal allocations for three common levels of risk aversion (Trustee/Market/Kelly Investor). Implement systematic backtesting: which includes both, regressing results of your portfolio on factors and study of the factors themselves (wrt the market excess returns).

Kinds of optimisation: mean-variance, Max Sharpe Ratio, higher-order moments (min coskewness, max cokurtosis) – implement at least two. Min Tracking Error also possible but for that your portfolio choice will be measured against a benchmark index. Computation by ready formula or specialised for quadratic programming. Adding constraints improves robustness: most investors have margin constraints / limited ability to borrow / no short positions.

OPTIONALLY, Risk Contributions can also be computed *ex ante* for any optimal allocation, whereas computing ERC Portfolio requires solving a system of risk budget equations (non-linear). ERC computation is not an optimisation, however can be ‘converted’ into one – sequential quadratic programming (SQP).

Portfolio Choice and Data

The choice of portfolio assets must reflect optimal diversification – according to your own understanding and approach. The simplest technical choice is choosing assets with the least correlation. For exposure/tilts to factor(s) – you need factor betas *a priori*, and include assets with either high or low beta, depending on the purpose specified by you.

A naive portfolio of S&P500 large caps can be said to be exposed to one factor, which is insufficient. A specialised portfolio for an industry, emerging market, credit assets should have 5-15 names (guide number), and > 3 notionally uncorrelated assets, such as commodity, VIX, bonds, credit spreads, real estate.

Factor time series (or several) can represent your uncorrelated asset – including factors as assets at the start and giving them preferential BL views is one way of systemic implementation of factor tilts.

- Mean-variance optimisation was specified by Harry Markowitz for simple returns (not log) which are *in excess* of the r_f . For risk-free rate, 3M US Treasury from pandas FRED dataset/ECB website rates for EUR/some small constant rate/zero rate – all are acceptable. Use 2-3 year sample, which means > 500 daily returns.
- Source for prices data is Yahoo!Finance (US equities and ETFs). Use code libraries to access that, Google Finance, Quandl, Bloomberg, Reuters and others. If benchmark index not available, equilibrium weights computed from the market cap (dollar value).
- In this variation of PC topic, it is necessary to introduce 2-3 factor time series and treat them as investable assets (5 Fama-French factors). If using Smart Beta ETFs present on their structure – you might find there is no actual long/short factors, just a long-only collection of assets with particularly high betas.

Step-by-Step Instructions

Part I: Factor Data and Study(Backtesting)

1. Implement Portfolio Choice based on your approach to optimal diversification. Usually the main task is to select a few assets that gives risk-adjusted returns the same as/outperforms a much larger, naturally diversified benchmark such as S&P500. See Q&A document distributed at the Workshop.
2. Experiment which factors you are going to introduce, collect their time series data or compute.
 - The classic Fama-French factors are HML (value factor) and SMB (small business). RMW (robust vs. weak profitability) and CMA (conservative vs aggressive capex) are the new factors and you can experiment with them.
 - Exposure to sector or style can also be considered a factor.
 - It very recommended that you introduce an interesting, custom factor such as Momentum, BAB (betting against beta) – likely you will need to compute time series of its returns, however that can be as simple as returns from a short portfolio of top five tech stocks.
3. The range of portfolios, for which factors are backtested, is better explained at source http://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html
4. Present P&L returns and Systematic Backtesting of your factors vs the Market (index of your choice), which includes performance, present plots of rolling beta and changing alpha. Ideally, you can present results for each factor beta independently and then, in combination. This work to be presented even before you engage in portfolio optimisation

Part II: Comparative Analysis of BL outputs

1. Plan your Black-Litterman application. Find a ready benchmark or construct the prior: equilibrium returns can come from a broad-enough market index. Implement computational version of BL formulae for the posterior returns.
2. Imposing too many views will make seeing impact of each individual view difficult.
3. Describe analytically and compute optimisation of **at least two kinds**. Optimisation is improved by using sensible constraints, eg, budget constraint, ‘no short positions in bonds’ but such inequality constraints $\forall w_i > 0$ trigger numerical computation of allocations. e.
4. You will end up with multiple sets of optimal allocations, even for a classic mean-variance optimisation (your one of two kinds). Please make your own selection on which results to focus your Analysis and Discussion – the most feasible and illustrative comparisons.
 - Optimal allocations (your) vs benchmark for active risk. Expected returns (naïve) vs implied equilibrium returns (alike to Table 6 in BL Guide by T. Idzorek.)
 - BL views are not affected by covariance matrix – therefore, to compute allocations shifted by views (through Black-Litterman model) with naive or robust covariance is your choice.

- Three levels of risk aversion – it is recommended that you explore at least for classical Min Var optimisation.
5. There is no rebalancing task for the project, particularly because posterior BL allocations expected to be durable.
 6. Compare performance of your custom portfolio vs factors and market (rolling beta), independently and jointly. OPTIONALLY, compare performance of your portfolio to $1/N$ allocations / Diversification Ratio portfolio / Naive Risk Parity kind of portfolio and perform the systematic backtesting of that portfolio *wrt* to factors.

Delta in a Local Volatility World. Optimal Hedging

Summary

We move to a serious quant footing for implied volatility, its surfaces, interpolation and generating process $\sigma(S, t)$. In 2025, there are options data vendors, including free data on the most liquid tickers. As quants, we can do more than descriptive statistics on options data. We *construct, regularize, and differentiate vol surface* and then, extract $\sigma_{\text{loc}}^2(S, t)$.

The surface construction techniques will help you to understand PDEs better, including the outcomes of Dupire Forward PDE, and how it is different from pricing under Backward PDEs (Black-Scholes). Forward equation is done in (K, T) but here we do not really evolve the surface: we interpolate and read off slope/curvature to compute local variance. That unlocks analytical derivatives on the surface ∂_T, ∂_K for hedging recipes. We also get density $f_{S_T} = \partial_{KK}$ for pricing. Outcome: a robust pipeline gives fits (derivatives) consistent with market vanillas and feeds pricing engines: we can bump, recompute Greeks for exotics, and automatically get the slope ∂_K to compute MVD analytically.

Sticky-strike vs sticky-delta

Local Volatility is known as ‘sticky-strike’ method, while traders consider ‘sticky-delta’ calibrations to be more effective. Our Min Variance Delta is such sticky-delta method. In this formulation, we will improve LV setup and work with moneyness, more relevant to sticky-delta. Another ‘kernel trick’ we invoke is to construct surface and derivatives on total variance $T \sigma_{\text{imp}}^2(T, y)$, which has a desired property of monotone-increasing function.

Numerical techniques of this topic will be applicable in other domains. They are (a) reformulation of PDEs (and more) with variable change, (b) derivatives computation and no-arb controls and (c), industrial-grade interpolation – monotone cubic spline, hermite segments and secants for slope, Akima spline. We will also cover *Andreasen-Huge* recipe that allows computing necessary (K, T) -derivatives from option prices directly. MV Delta estimation revisited as well, for which you can optionally try recursive weighted LS or state-space filtering for stability of $\hat{a}_t, \hat{b}_t, \hat{c}_t$ coefficients.

Options Data

- **OptionsDX** — free historical options data for major tickers (SPX, SPY, TSLA, QQQ) with strikes, expiries, bid/ask, sizes, and Greeks. Good and current choice (2025) that provides some free End Of Day (EOD) option quotes.
- **Polygon.io** — commercial API with quotes and reference data; useful if you already use Polygon for equities/crypto.
- **Thetadata** — robust dataset, daily refresh and live quotes suitable for backtesting; REST docs and quick-start available.
- **Dolthub repo** — SQL/Git-style versioned options data you can query directly.

The project does not require Bloomberg/FO level option price data but you can certainly use any relevance source – once you moved to your own grids and interpolation the data should not be classed as provider’s. Implied Vols can also be drawn from a uniform distribution (empirically set range, eg 25-35%

for delta/maturity bucket) – while that definitely disturbs your surfaces (or makes them less realistic) it's acceptable to generated data, or generate where the original data has gaps.

By-Step on Local Volatility: Surface and No-Arb (Part I)

1. From quotes data to grids. Start from quotes in tuples $T_i, K_j, \sigma_{ij}^{\text{mkt}}$; compute forwards F_{T_i} for which you need U.S. Treasury daily par yield curves (CBOE methodology). Then decide to work in K_j or log-moneyness $y_{ij} = \ln(K_j/F_{T_i})$. Most important to compute the total variance $w_{ij} = T_i(\sigma_{ij}^{\text{mkt}})^2$
 - Regularise y -grid per term for interpolation and derivatives; keep the native K -grid for pricing checks and no-arb tests.
2. Across skew (per T_i) interpolation: build a convex slice delicately. Encouraged to fit a model-driven skew parameterisation in y or do piecewise-linear prices in K with convexity constraints. Map for consistency among grids $w(T_i, y)$, $\sigma_{\text{imp}}(T_i, K)$, and $C(T_i, K)$. Control so that the induced curvature remains non-negative in the wings of skew.
3. Across time (per fixed y): enforce calendar monotonicity, eg $w(T_{i+1}, y) \geq w(T_i, y)$. Interpolation of total variance $w(T, y)$ is linear in T . Techniques are C^1 PCHIP monotone cubic spline, hermite segments and secants for slope, Akima spline. Smoothing can be added on top of interpolation but we shouldn't need that.

By-Step on Local Volatility: Actual σ_{LV} (Part II)

4. Dupire ingredients are derivatives we take over the grid – in order to ultimately compute $\sigma_{\text{loc}}^2(K, T)$. We have two grid choices $w(T_i, y)$ and $C(K_j, T_i)$ to compute derivatives. y has no index because grid to be regularised.
5. Derivatives are computed by finite differences and your quant technique here is about two decisions: where to increase the order of differencing and where to invoke modified differencing formulae known as 'boundary stencils'.
Produce stability checks, summary of repairs for no-arb (calendar/butterfly), and where you boundary/higher order stencils for repair.
6. Compute $\sigma_{\text{loc}}^2(K, T)$ using the formula recommended in Project Workshop.

Andreasen-Huge local volatility from option prices (ALTERNATIVE I & II)

- A smooth price surface $C(T, K)$ can be constructed from data or converted from $w(T_i, y)$ surface. Techniques are covered in Part I: interpolate *prices* with slicewise convexity in K and linear in $\sqrt{T}$ – notice this square root over time.
- Under this recipe, local variance computed using Dupire Forward PDE,

$$\sigma_{\text{LV}}^2(K, T) = \frac{\partial_T C + (r - q) K \partial_K C + q C}{\frac{1}{2} K^2 \partial_{KK} C}$$

$\sigma_{\text{loc}}^2(K, T)$ can always be mapped to $\sigma_{\text{LV}}(S, t)$ by setting $S = K, t = T$ – the question is that our (K_j, T_i) -grid has all values necessary for the states of S, t .

By-Step on Minimum Variance Delta (Part III)

1. With the detailed Local Volatility implementation, the derivative $\frac{\partial \sigma_{imp}}{\partial K}$ is known from our Part II work on Dupire. It is a slope – how fast the surface tilts if you nudge strike. δ_{MV}^{LV} can be computed. We will also have Vega smarter than Vega_{BS} .
 2. δ_{MV} estimation via $a + b\delta_{BS} + c\delta_{BS}^2$ is optional for FP derivation but you can still compute sparsely, for buckets $0.1 < \delta_{BS} < 0.9$ and say two maturity buckets $1M-2M$ and $5M-6M$. This empirical estimation gives you opportunity to try Recursive LS or state-space filtering for stability of $\hat{a}_t, \hat{b}_t, \hat{c}_t$ coefficients (sensitivity study and validation).
 3. Compute Gain as recommended (HW 2017) with estimated δ_{MV} and theoretical derivative-based δ_{MV}^{LV} (comparison study of Gain). **END OF BY-STEP**
-

More on Local Volatility / LV Topic Deliverables summarised

- In our LV setup we change variable to moneyness, interpolate, compute finite differences and then return to local variance result.
- We have an arbitrage-free Total Variance surface with industry-scale interpolation choices. Also could have surface of prices $C(T, K)$. Produce tables and figures of $\partial_T C$, $\partial_{KK} C$.
- Produce stability checks, summary of repairs (calendar/butterfly) for no-arb, and where you chose non-uniform grid and boundary stencils /higher order stencils for repair.
- Local volatility surface $\sigma_{LV}(S, t)$ on a production grid;
- Comparison between Local Vol and Black-Scholes vol can be done quickly by running the regression and evaluating β as advised in Workshop slides.

Uses for local volatility. **A.** After remapping you can use $\sigma(S, t)$ in Backward FDE finite-differencing for prices, in Monte-Carlo simulations. **B.** You can stress the surface in T and K (or y) and reprice exotics. These can be either sticky-strike or sticky-delta shifts. Essentially you generate scenario P&Ls which are smile-consistent. You can compare LV skews $\frac{\partial \sigma_{imp}}{\partial S}$ to parameters of Heston's stochastic volatility as well as to realized variance. **C.** Don't forget the forward density f_{S_T} which you can read off the surface and use for quantiles, Value at Risk, tail probability and Expected Shortfall.

There is no closed-form, Black-Scholes style formula for **Vega** from Local Volatility. Below are two routes for computation of **s-Vega** (surface Vega).

- Andreasen-Huge: price is linear in $\sqrt{T}$ between maturities and convex in K per skew, so sensitivities to pillar prices are analytic weights; price bumps can be mapped to implied-vol bumps via a Jacobian.
- Alternatively, bump the total variance function $w(T_i, y_j)$ and use adjoint/FD to compute $\partial V / \partial w_{ij}$.

Blending Ensemble for Classification

Summary

Trend prediction is valuable for investment management as accurate prediction could ensure asset managers outperform the market. Trend predictions can be modeled effectively using machine learning algorithms; however, the choice of learning techniques to be adopted remains a challenging task. Ensemble learning is a powerful machine learning algorithm that combines the predictions of multiple machine learning models by mitigating the errors or biases that may exist in individual models by leveraging the collective intelligence of multiple models that leads to a more precise prediction.

Objective

Your objective is to produce a model to predict positive moves (up trend) using the Blending Ensemble technique. Your proposed solution should be comprehensive with the detailed model architecture, evaluated with a backtest applied to a trading strategy.

- Choose one ticker of your interest from the index, equity, ETF, crypto token, or commodity.
- Predict trend only, for a short-term return (example: daily, 6 hours). Limit prediction to binomial classification: the dependent variable is best labelled $[0, 1]$. Avoid using $[-1, 1]$ as class labels.
- Analysis should be comprehensive with detailed feature engineering, data pre-processing, model building, and evaluation.
- Use only machine learning algorithms for this project.

Note: You are free to make study design choices to make the task achievable. You may redefine the task and predict the momentum sign (vs return sign) or direction of volatility. Limit your exploration to ONLY one asset. At each step, the process followed should be expanded and explained in detail. Merely presenting python codes without a proper explanation shall not be accepted. The report should present the study in a detailed manner with a proper conclusion. Code reproducibility is a must and the use of modular programming approaches is recommended. Under this topic, you do not recode existing indicators, libraries, or optimization algorithms.

Step-by-Step Instructions

1. The problem statement should be explicitly specified without any ambiguity including the selection of underlying assets, datasets, timeframe, and frequency of data used.
 - If predicting short-term return signs (for the daily move), then training and testing over up to 5 years should be sufficient. If you attempt the prediction of 5D, 10D return for equity or 1W, 1M for the Fama French factor, you'll have to increase the data required to at least 10 years.
2. Perform exhaustive Feature Engineering (FE).
 - FE should be detailed including the listing of derived features and specification of the target/label. Devise your approach on how to categorize extremely small near-zero returns (drop from the training sample or group with positive/negative returns). The threshold will strongly depend on your ticker. Example: small positive returns below 0.25
 - Class imbalances should be addressed - either through model parameters or via label definition.
 - Use of features from cointegrated pairs and across assets is permitted but should be tactical about design. There is no one recommended set of features for all assets; however, the initial feature set should be sufficiently large. Financial ratios, advanced technical indicators including volatility estimators, and volume information can be a predictor for price direction.
 - OPTIONAL Use of news heatmap, credit spreads (CDS), historical data for financial ratios, history of dividends, purchases/disposals by key stakeholders (director dealings) or by large funds, or Fama-French factor data can enhance your prediction and can be sourced from your professional subscription.
3. Conduct a detailed Exploratory Data Analysis (EDA).
 - EDA helps in dimensionality reduction via a better understanding of relationships between features and uncovers underlying structure, and invites detection/explanation of the outliers. The choice of feature scaling techniques should be determined by EDA.
4. Proper handling of data is a must. The use of a different set of features, lookback length, and datasets warrant cleaning and/or imputation.
5. Feature transformation should be applied based on EDA.
 - Multi-collinearity analysis should be performed among predictors.
 - Multi-scatter plots presenting relationships among features are always a good idea.
 - Large feature sets (including repeated kinds, and different lookbacks) warrant a reduction in dimensionality in features. Self Organizing Maps (SOM), K-Means clustering, or other methods can be used for dimensionality reduction. Avoid using Principal Component Analysis (PCA) for non-linear datasets/predictors.

6. Perform extensive and exhaustive model building.

- The architecture of a stacking model should involve at least three base learners.
- Hyperparameters of each base learners should be optimized.
- Type of base learners and meta model to be used are design choices.

7. The performance of your proposed classifier should be evaluated using multiple metrics including back-testing of the predicted signal applied to a trading strategy.

- Investigate the prediction quality using AUC, confusion matrix, and classification report including balanced accuracy.
- Predicted signals should be evaluated by applying them to a trading strategy.

* * *

Algorithmic Trading for Reversion and Trend-Following v2025-2

Algorithmic trading (intraday execution and short-horizon alpha) is concerned with how to optimally run strategies, which are deceptively simple in maths. Set aside technical execution/CPU computation, the traders look in-depth at order types as allowed by the exchange, time-in-force, incorrect messages/misleading data from a broker, and own data storage (15 Min and more frequent). However and first task for this quant's project is to algo-adapt and back-test: (a) two kinds of trend-following strategy with several indicators of your choice, (b) one kind of non-trivial reversion strategy (no simple Z-scores), and [recommended] (c) one kind of alternative strat with the use of encoder/classifier.

Extend the strategies from backtesting to a live-testing – scripts with Broker API of your choice, which can be of simple scope. The trading code must handle exceptions and inconsistent/incorrect information received back from the broker, such as order filled when it wasn't. For the purposes of FP, we intentionally don't consider more complex issues, such as market impact, scaling of a strategy with capital. Real-time trading relies on loops/crone jobs which check for buy/sell signal. To improve performance more optimized (than Python) languages used in the industry, eg GoLang.

Towards Robust Algotrading

- Indicators. Experiment with your indicators: mathematical, temporal variations. There are implied periods for averaging / resampling interval – to see what is feasible.
- Indicators. Modify your indicators to be event-time as opposed to purely price-based. For example, volume-weighted indicators reduce microstructure noise. Build bars by a fixed number of trades (tick bars), a fixed volume target (volume bars), or a fixed notional target (dollar bars). Rolling windows can be computed *event counts* (not minutes).
- Robustness. Extend your backtesting with hypothesis testing. Use the tools of Information Coefficient $IC = \text{corr}(f_t, r_{t+1})$. When you have too many features, use AIC/BIC. Use Deflated Sharpe and alike for performance evaluation.
- Slippage Try to decompose slippage, what contributes to it and remember order costs.

Part I: Generic Strategies Made Proprietary

Implement *your design* of core strategies – backtesting can be done Python on historical data. While you can use libraries like TA-Analysis, Pyfolio Reloaded, Python has no single ready-made solution.

Full mathematical description of indicators chosen and your experiments, eg the ratios over different timeframes must be given. Understanding mathematical and temporal nuance of indicator. Also the same indicator (eg ADX, MACD) can be used for different purposes. No need to provide details of inferential, hypothesis testing and/or statistical learning maths.

1. For a trend-following strategy type, common choices are Exponential Moving Average (EMA) and Average Directional Index (ADX), an oscillator. Other approach is a convergence/divergence indicator, MACD. For each strategy, you need to decide on: your

indicator/combination thereof/formula modification with Volume (VWAP), and what constitutes a trading signal (eg, crossover of EMA_{20D} with another).

Simple but practical trend indicator primarily used in FX is of the following design:

Steps 1 & 2: Sample the prices at regular intervals (30 sec) for price level or average; experiment over which longer period exactly to take an average (eg, 5-minute intervals). Can use `DataFrame.resample`.

Step 3: Compute the ratio of price (or short-average) to long-term average: near 1 signals 'no trend' as short-term prices $\approx$ the long-term prices. Uptrend is signaled by the ratio above 1, and downtrend by less than 1. Give several calibrations.

2. For a mean-reversion strategy type,

Steps 1 & 2: Z^* deviation from the price (over very short horizon) is likely a bad signal. Consider other measures of distance (statistical learning). Consider OU process (Topic TS) only if mean-reversion is a reasonable expectation over your traded horizon.

3. For an alternative strategy/encoders use [recommended but optional], you can choose to tackle one of the following problems

Order book Typical application is for encoder is to compress Level II data (order-book data) for features and enable realistic scenario generation. Variational autoencoder is trained via the evidence lower bound (ELBO) recipe.

Problem *Irregular sampling and multi-horizon conditioning*. Encoder-only Transformers (special deep learning NN architecture) handle missing ticks and variable horizons more robustly; use event-time bars and standardized features (eg, Z-score with EWMA $\lambda = \exp(-\ln 2/h)$, and set half-life h experimentally).

Problem Small encoders meet live 'latency budgets' more easily than heavy sequence-to-sequence models.

Part II: Broker API and Input data

1. Treat this project as more professional, eg, even for a historical backtesting fetch data from OpenBB/brokerage (vs Yahoo!Finance) and write a couple of routines checking data quality. You are encourage to work with 15-minute and higher-frequency data.

2. The common API choice is REST (Representational State Transfer).

- Alpaca, Interactive Brokers Web, and Oanda all have their own versions. In particular, Alpaca REST API is free and includes asynchronous events handling based on WebSocket and Server Side Events (SSE).
- REST API is referred to as HTTP API because it utilizes HTTP methods, eg GET (retrieve data), POST (create new data), PUT (update data). Useable for non-time-critical operations such as retrieving historical data, account information, placing orders, and getting order status.

3. The more industrial strength and lower latency API choice is FIX (Financial Information eXchange). A messaging protocol designed for the **real-time** exchange of securities transactions. It supports submission and cancellation of various order types, trade execution reports, and market data dissemination – all for high frequency. FIX is used by large institutions, funds, and broker/dealers.

4. As an example only: Interactive Brokers TWS API allow different languages C++, C#, Java, Python.

Describe order types suitable to your tickers and strategies, and attempt code for order loops and order handling using an API. Set price parameters far from the market to avoid execution. It is absolutely recommended that you do not run any actual live-trading.

Part III: Evaluate Risk and Test Thrice

An opened market position is exposed to various types of risk. While the market risk can be managed with quant methods like rolling VaR, order-handling risk is more important and reliant on API choices (Part II) as well as dev environment and stack of libraries choices.

1. Containerise and consider docker and crone-style scheduling (think how different parts of your code can be scheduled to run as an independent scripts). Consider implications for running the image of your code data collection, backtest, and trading order loops on a virtual server.
2. Positions Tracking. Code can request account updates (eg, in loop) to provide a secondary confirmation layer.
 - Code must have a **verification** of server responses. From the broker side orders may be partially filled and/or returned with an incorrect fill information (eg, ticker not bought when it was actually bought).
 - Catch the stale websockets and connection issues.
3. Performance and Risk Reporting. Of less utility to algo trading are *Pyfolio*-style systematic backtesting, scorecards with concentration in asset vs portfolio, Beta-to-SPY. However, do report on *P&L* performance, turnover, trading costs, and strategy-specific drawdowns.
 - One recipe but not mandatory is to present a risk dashboard: run a strategy alike to live for N trading days; save fills and *P&L* in TimescaleDB (your own database); display drawdown, Sharpe, intraday VaR. You can try *Streamlit*.
4. Market Data. Introduce simple checks to catch inconsistencies in market data (downloaded for a backtest or received from the broker live). For example, futures price can't be below spot price, implied quantities can't be negative.

The general principle is to preserve the trading capital as much as possible. Large algorithmic traders, market-makers achieve that through very robust systems that control risks.

Deep Neural Networks for Solving High Dimensional PDEs in Quantitative Finance (DeepPDE)

OVERVIEW

There is little doubt that deep learning techniques have revolutionized certain areas of computer science such as computer vision and natural language processing. However, the impact of deep learning and other machine learning techniques has been limited in quantitative finance, especially in traditional areas of quantitative finance such as option pricing, hedging and portfolio management. These traditional areas of quantitative finance are dominated by rigorous mathematical models, and pricing frameworks such as risk-neutral evaluation. In contrast computer science applications rely on high quality empirical data. As a result we have seen deep-neural networks be used to mainly solve regression problems in order to develop fast pricing engines (e.g. a neural network is given the prices of various contracts and learns the pricing function). However in the last few years we have observed a number of approaches that go beyond the simple function fitting/regression framework and several recent works have embraced the new set of tools. The objective of this project is to investigate the use of Deep-Neural Networks for solving high-dimensional PDEs that appear in pricing and risk management applications.

METHODOLOGY

In this project you will develop a Deep Neural Network (DNN) that can solve the following class of d -dimensional Partial Differential Equations:

$$\begin{aligned} \frac{\partial u}{\partial t} + \frac{1}{2} \text{Tr}(\sigma(t, x) \sigma(t, x)^\top \nabla^2 u(t, x)) + \nabla u(t, x)^\top \mu(t, x) + f(t, x, u(t, x), \sigma(t, x)^\top \nabla u(t, x)) &= 0 \\ u(T, x_1, \dots, x_d) &= g(x_1, \dots, x_d) \end{aligned} \quad (1)$$

Where:

- $u(t, x)$ with $t \in \mathbb{R}_+$, and $x \in X \subset \mathbb{R}^d$ is the unknown function we need to compute, $\nabla u(t, x)$ is the gradient of the solution with respect to x and $\nabla^2 u(t, x)$ is the Hessian (again with respect to x).
- $\sigma(t, x)$ is a known $d \times d$ matrix valued function.
- Tr denotes the trace of a matrix and T denotes a transpose.
- f is a known nonlinear function
- g is a known boundary condition.

While it is entirely possible to develop an approach to solve the general case in (1) your starting point for this project is the Black-Scholes PDE with d uncorrelated assets and with a constant volatility and risk free rate. You are encouraged to first understand the provided code for this problem before extending it to another pricing problem. Consider the following special case of (1) that can be used to price European-style contracts,

$$\begin{aligned} \frac{\partial u}{\partial t} + \sum_{i=1}^d \left(\frac{1}{2} \sigma_i^2 \frac{\partial^2 u}{\partial x_i^2} + r \frac{\partial u}{\partial x_i} x_i \right) - ru(t, x) &= 0 \\ u(T, x_1, \dots, x_d) &= g(x_1, \dots, x_d) \end{aligned} \quad (2)$$

Extensions to other type of contracts and other applications of the framework (e.g. for solving stochastic optimal control problems are given at the end of this document).

In this project we will focus on one of the most promising approaches to solving (2) that relies on solving a forward-backward system of stochastic differential equations. The derivation of the equations below will also be explained in the project workshop.

$$\begin{aligned} dX_t^i &= rX_t^i dt + \sigma^i X_t^i dW^i, \quad X_0^i = x^i, \quad i = 1, \dots, d \\ dY_t &= rY_t dt + \sum_{i=1}^d \sigma^i X_t^i Z_t^i dW^i, \quad Y_T = g(X_T^1, \dots, X_T^d) \end{aligned} \quad (3)$$

The interpretation of (3) is as follows: X^i denotes the price of asset i and for simplicity we use a constant $r > 0$ risk free rate. Note that for simplicity we have also assumed that $\sigma^{i,j} = 0$ for $i \neq j$ and that the Brownian motions are all uncorrelated i.e. $\text{cov}(dW^i dW^j) = 0$ for $i \neq j$. The second equation for Y is a Backward Stochastic Differential Equation. The unknowns for the BSDE are Y_t and Z_t . In terms of interpretation it can be shown that $Y_t = u_t$ and $Z_t = \nabla_x u(t, x)$. In other words Y_t is the price of the option and Z_t is the option's delta.

Since Z_t is not known it is not possible to (easily) simulate the system in (2). Instead the idea is to parameterize the solution $u(t, x) \approx u(t, x; \Theta)$ where Θ are the parameters of a neural network, and compute $Z_t \approx Z_t(\Theta) = \nabla_x u(t, x; \Theta)$. The derivative $\nabla_x u(t, x; \Theta)$ is with respect to x and can be computed with backpropagation.

STEP-BY-STEP INSTRUCTIONS

Your code should have the following main steps.

1. Define a Neural Network to represent $u(t, x; \Theta)$. See the example code provided for an example of how to set this up using Pytorch.
2. Generate M Monte Carlo paths using a simple Euler scheme to discretize the dynamics in (3). Since your Euler discretization will use $Z_t(\Theta)$ instead of the correct Z_t you will need to optimize the parameters Θ
3. Use an optimizer to optimize the Neural Network parameters. Define an appropriate loss function such as the one described in [2].

A minimum working example using the Pytorch library is provided along with the specification. The minimum working example prices vanilla call options, you should modify/extend the code to solve other problems. A successful project should: be able to demonstrate an extension of the provided code to high dimensions (e.g. $d = 100$) and (optionally) apply the framework to a different product class than the one provided. Below are some ideas and references you can use to extend your code.

- The working example is based on the work from [2]. In the paper you can find some ideas on how to train the model faster and some more stable architectures that work for very high dimensions $d = 100$.
- The article in [3] is also a good introduction to the topic and has a discussion on exotic options (still in one-dimension).
- The article in [4] reviews several architectures and algorithms for solving high-dimensional pricing problems.
- The article in [1] investigates the use of deep neural networks for XVAs.
- Feel free to explore the papers above and other references and consider extending the minimum working example to applications that are of interest to you, e.g. introduce correlation between the assets, consider dynamics that are not log-normal, price exotics or XVAs, etc.

References

- [1] A. Gnoatto, A. Picarelli, and C. Reisinger. Deep xva solver: A neural network-based counterparty credit risk management framework. *SIAM Journal on Financial Mathematics*, 14(1):314–352, 2023.
- [2] B. Güler, A. Laignelet, and P. Parpas. Towards robust and stable deep learning algorithms for forward backward stochastic differential equations. *33rd Conference on Neural Information Processing Systems (NeurIPS 2019), Vancouver, Canada.*, 2019.

- [3] B. Hientzsch. Deep learning to solve forward-backward stochastic differential equations. *Risk Magazine*, February, 2021.
- [4] J. Liang, Z. Xu, and P. Li. Deep learning-based least squares forward-backward stochastic differential equation solver for high-dimensional derivative pricing. *Quantitative Finance*, 21(8):1309–1323, 2021.

ELECTIVES RECOMMENDATION - CQF June 2025

Credit Spread for a Basket Product	Counterparty Credit Risk Modeling Numerical Methods R for Data Science and ML
Delta in a Local Volatility World. Optimal Hedging	Advanced Volatility Modeling Numerical Methods
Portfolio Construction using Black-Litterman	Risk Budgeting for Asset Allocation Advanced Portfolio Management Behavioral Finance for Quants
Pairs Trading Strategy Design & Backtest	R for Data Science and ML Energy Trading Algorithmic Trading II
Deep Learning for Asset Prediction	Advanced Machine Learning I Advanced Machine Learning II
Blending Ensemble for Classification	Advanced Ensemble Modeling Advanced Machine Learning II
Algorithmic Trading for Reversion and Trend-Follow	Algorithmic Trading I Algorithmic Trading II Decentralized Finance Technologies
DNNs for Solving High Dimensional PDEs	Advanced Machine Learning I Advanced Volatility Modeling

Reading List for Final Project v2025-2

These are curated readings for each topic. Papers give a scheme/focus on either pricing model or techniques/statistical tests Whenever possible, readings will be distributed with with Project Workshops and/or Project Tutorials in files *Topic XX - Additional Material.zip*. Otherwise please check SSRN and your alternative sources.

Reading List: Local Volatility. Optimal Hedging (LV, DH)

- FP Project workshop will cover Forward PDE and local volatility computation. There will also be further technical guidance on ‘quality control’ for the construction of surfaces from option prices and implied vol.
 - Exercises/Solutions in 3.10 Advanced Volatility Modelling – issued 2025-Oct [NEW].
 - Python Labs *Finite Difference Methods* and *Implied Volatility* – exact titles can vary.
- *Optimal Delta Hedging for Options* by John Hull & Alan White. *Journal of Banking and Finance*, Sep 2017, papers.ssrn.com/sol3/papers.cfm?abstract_id=2658343.
- *Lectures 6/7: Local Volatility* by Emmanuel Derman (2008) – these lecture slides are not a concentrated reading, more introductory.
- *The Volatility Surface: A Practitioner’s Guide* by Jim Gatheral (2012) is a good source and but paper-based book.

Reading List: Cointegrated Pairs (TS)

- Cointegration Lecture introduces multiple tests for stationarity, each to their own purpose. ADF is simple but exercise care about critical values from software. *Critical Values for Cointegration Tests* by J MacKinnon (2010) explains the compute; there is an additional note on *Mathematics of Dickey Fuller Test*.
 - *Tutorial: Nonstationarity* which is a useful R notebook.
 - *Tutorial: Cointegration in Rates - Multivariate Cointegration* another up to date R notebook (2023).
 - *Efficient Pair Selection for Pair-Trading Strategies* by P. McSharpy applies ADF test to filter and search for coint pairs in the market.
- *Explaining Cointegration Analysis: Part I* by D. Hendry & K. Juselius can be read instead of an econometrics textbook. Part II concerns with VECM detail and deterministic terms.
- *Learning and Trusting Cointegration in Statistical Arbitrage* by R. Diamond (2014), *WILMOTT* go directly to Appendix on the OU process maths, and Appendix on number of trades.

Reading List: Credit Portfolio (CR)

- *CDO & Copula Lecture* material is for reference, slides 48-52 illustrate Elliptical copula densities and Cholesky factorisation.
- *Sampling From Copula* algorithm is in *Project Workshop*. See Ch 5 of *Monte Carlo Methods in Finance* Peter Jaekel (2002) for a schema as well.
- There is a special note on *Topic CR Q&A*, which includes Rank Correlation.

Reading List: Portfolio Construction (PC)

- CQF Lecture on *Fundamentals of Optimization and Application to Portfolio Selection*
- *A Step-by-step Guide to The Black-Litterman Model* by Thomas Idzorek, 2002 gives the schema to the BL model implementation.
- *The Black-Litterman Approach: Original Model and Extensions* Attilio Meucci, 2010.
<http://ssrn.com/abstract=1117574>

Reading List: Deep Learning and Ensemble Learning (DL, ML)

- *Short-term stock market price trend prediction using a comprehensive deep learning system* by Jingyi Shen and M. Omair Shafiq, Journal of Big Data, Vol 7 (2020).
- *A graph-based CNN-LSTM stock price prediction algorithm with leading indicators* by Jimmy Ming-Tai Wu et al. (2021).
- *A comprehensive evaluation of ensemble learning for stock-market prediction* by Isaac Kofi Nti et al., Journal of Big Data, Vol 7 (2020).
- *A Blending Ensemble Learning Model for Crude Oil Price Prediction* by Mahmudul Hasan et al. (2022). papers.ssrn.com/sol3/papers.cfm?abstract_id=4153206.

Reading List: Solving High-Dimensional PDEs (DN)

- B. Hientzsch. *Deep learning to solve forward-backward stochastic differential equations*. Risk Magazine, February, 2021.
- J. Liang, Z. Xu, and P. Li. *Deep learning-based least squares forward-backward stochastic differential equation solver for high-dimensional derivative pricing*. Quantitative Finance, 21(8):1309-1323, 2021.

Reading List: Algorithmic Trading (AL)

- *Algorithmic Trading Workshop Notes* apply to both sessions, but things have evolved since October 2023 workshop delivery.
- *Algo Trading I* and *Algo Trading II* give Python code examples – *alpaca*, *backtesting*, *timescale* still relevant, but *openbb* access changed.
- *Data Sources for Options, Equities and Factors* has useful updates in `S2_Data_Equities_Factors v2.html`. You can also reuse Python Labs that work with Time Series.
- F. Barez, P. Bilokon, A. Gervais, N. Lisitsyn *Exploring the Advantages of Transformers for High-Frequency Trading*, February 2023, available at <https://arxiv.org/abs/2302.13850>.